

Zobrist Hashing: the splitmix64 Bijection and Incrementality

Theorem monograph T09 of the chess-dynamics-lab program —
the KLEIN layer, memory of the flow

Isaev Iskhak Khamzatovich

Chess Particle Dynamics Laboratory · chess-dynamics-lab

Keywords: Zobrist hashing, splitmix64, bijection, incrementality, birthday bound

Zobrist Hashing: the splitmix64 Bijection and Incrementality

Theorem monograph T09 of the chess-dynamics-lab program — the KLEIN layer, memory of the flow

Isaev Iskhak Khamzatovich

Chess Particle Dynamics Laboratory · chess-dynamics-lab

Keywords: Zobrist hashing, splitmix64, bijection, incrementality, birthday bound

Problem statement

The KLEIN layer is the memory of the model: the discrete flow of moves must distinguish its states, otherwise the transposition tables, the retrograde analysis T11 and the whole verification protocol lack a common coordinate system. The classical solution is Zobrist hashing (1970): each atomic component of the state receives a random 64-bit key, and the hash of a position is the XOR of the keys of all components present. The XOR composition is cheap and incremental, but its correctness rests on two facts that must be proved: the key generator must produce a reproducible and statistically sound sequence without external entropy sources, and the incremental update must coincide exactly with the full recomputation at every step of the flow.

We answer both requirements constructively. The generator is splitmix64, the finalizer of the FastHash family: an addition of the golden constant followed by two rounds of xorshift-multiply. It is compact enough that the same function in the seven languages of the polyglot core produces bit-identical tables: cross-language reproducibility becomes part of the theorem rather than a hope for port accuracy. Incrementality is proved by induction over the move sequence with an explicit delta formula covering castling, en passant and promotion.

A separate question is the safety of the 64-bit space: is it sufficient for a transposition table. We give the exact birthday bound and show that even with a billion stored positions the probability of at least one collision stays below three percent, while the practical regimes of the laboratory are orders of magnitude deeper inside the safe zone.

Theorem

Theorem T09 (splitmix64 bijectivity and hash incrementality). The splitmix64 map: $x0 = x + \text{gamma}$, $\text{gamma} = 0x9E3779B97F4A7C15$; $x1 = (x0 \oplus (x0 \gg 30)) \cdot 0xBF58476D1CE4E5B9$; $x2 = (x1 \oplus (x1 \gg 27)) \cdot 0x94D049BB133111EB$; $\text{sigma}(x) = x2 \oplus (x2 \gg 31)$; all operations modulo 2^{64} . Then: (i) sigma is a bijection of $\mathbb{Z}/2^{64}$ with an explicitly writable inverse; (ii) the key table of 1562 words seeded by $0x1234567890ABCDEF$ consists of 12 x 128 piece keys, one side key, 16 castling keys and 9 en-passant keys; (iii) for any position P and move m the incremental hash equals the full recomputation: $h(\text{make}(P,m)) = h(P) \oplus \Delta(m)$, where $\Delta(m)$ depends only on the components changed by the move; (iv) under random keys the collision probability for n positions is at most $n(n-1)/2^{65}$; for $n = 10^9$ this is 0.027.

Proof

(i) Bijectivity. All three stages are bijections of the ring $\mathbb{Z}/2^{64}$. Addition of a constant is trivially bijective. Multiplication by a constant u is bijective exactly when u is odd: an odd u is coprime to 2^{64} , so by Bezout there exists $u^{-1} \bmod 2^{64}$, and both constants $0xBF58476D1CE4E5B9$ and $0x94D049BB133111EB$ are odd. The xorshift $x \rightarrow x \oplus (x \gg s)$ is inverted by the infinite series $y \rightarrow y \oplus (y \gg s) \oplus (y \gg 2s) \oplus \dots$: the high bits of x are recovered immediately, the low ones by cascade, and the series terminates once the shift reaches the word width. For s in 30, 27, 31 two terms s and $2s$ suffice: the term $y \gg 3s$ and all later ones vanish since $3s \geq 64$. The inverse function is implemented literally: the cascades are unwound in reverse order, the odd multipliers are replaced by modular inverses, and finally gamma is subtracted.

Bijectivity is confirmed constructively: the round-trip $\text{sigma}^{-1}(\text{sigma}(x)) = x$ is executed for all x in $[1, 2000]$ in check C7 and for all 1562 table keys in C9. The first values of the sequence fix the cross-language reference: $\text{sigma}(1) = 0x910A2DEC89025CC1$, $\text{sigma}(2) = 0x975835DE1C9756CE$, $\text{sigma}(3) = 0x1D0B14E4DB018FED$. Each of the seven polyglot backends must reproduce them bit for bit; a mismatch would indicate a defect of integer arithmetic — a signed shift, loss of high bits without 64-bit unsigned types, or wrong overflow emulation.

(iii) Incrementality. Induction over moves. Define $h(P)$ as the XOR of the keys of all atoms of the position: one key per piece on a square, plus the side key (when Black is to move), the castling-state key (out of 16) and the en-passant file key (index 0 = none). Let $\Delta(m)$ be the XOR of the keys of all atoms changed by the move m : the piece leaving the source square, the piece arriving at the target, the captured piece, the pawn captured en passant, the promotion, the rooks in castling, the old and new castling states, the old and new ep keys, and the side key twice. Every removed atom enters $h(P) \oplus \Delta(m)$ an even number of times and disappears, every added one enters an odd number of times and appears; atoms not touched by the move do not enter Δ at all. The base of the induction is the initial position, the step is an arbitrary legal move; castling and en passant are handled by separate branches

(four rook cases, the ep index being the file plus one rather than a square). Hence the incremental hash after make coincides with the full recomputation at every step.

Check C9 executes 40 random games of up to 60 plies (the fixed seed 42): after every ply the incremental hash is compared with the recomputed one — no mismatches; additionally, unmake of each of the first eight moves restores both the FEN and the hash of the start position exactly. Note a historical detail fixed by the theorem as part of the definition: indexing the ep key by square instead of file would produce false collisions inside one file — the file indexing was corrected and pinned by the protocol.

(iv) Collision bound. For n independently and uniformly chosen 64-bit values the probability of at least one collision is at most $\text{binom}(n,2)/2^{64} = n(n-1)/2^{65}$ by the union bound. For $n = 10^9$ this is $10^{18}/2^{65} \approx 0.0271$; for $n = 10^6$ it is $2.7\text{e-}8$. The practical regimes of the laboratory — search to depth 6 and retrograde bases of about $4\text{e}6$ states — lie far inside the second regime. The theorem is proved.

Protocol data

Reference values of splitmix64 (cross-language certificate)

Quantity	Value
splitmix64(1)	0x910A2DEC89025CC1
splitmix64(2)	0x975835DE1C9756CE
splitmix64(3)	0x1D0B14E4DB018FED
key table seed	0x1234567890ABCDEF
golden constant	0x9E3779B97F4A7C15

Composition of the key table (1562 words in total)

Group	Number of keys	Indexing
piece	1536	type-color $\times$ 0x88 square
side	1	XOR when Black is to move
castling	16	bitmask of rights
en passant	9	0 = none, 1..8 = file + 1

Results of check C9

Parameter	Result
incrementality (40 games $\times$ $\leq$ 60 plies)	full agreement with recomputation
unmake restoration	FEN and hash exact after every move
inverse round-trip	$x = 1..2000$, no exceptions
collision bound at $n = 10^9$	0.0271

Summary of what is proved

We proved the bijectivity of splitmix64 with an explicit inverse, fixed the composition and the seed of the 1562-key table as a cross-language reference, proved the incremental identity $h(\text{make}(P,m)) = h(P) \oplus \text{Delta}(m)$ by induction over moves with an explicit delta, and gave the exact collision bound. The KLEIN layer thereby received a reliable memory: transposition tables, the retrograde bases of T11 and the determinism of the search T10 operate on a hash whose correctness is certificate-verified and reproducible in seven languages.

Verification

python3 dynamics.py -run C9 (incrementality, unmake, the inverse round-trip);
python3 dynamics.py -run C7 (the inverse function on the segment 1..2000); the
C9 line of the polyglot core in all seven languages must print [PASS] with the same
reference constants. The C9 runtime is a fraction of a second even on a phone.